SDI Investment Property Marketplace

Feature Test Plan

What to click, what should happen, and what each result proves

Scope	The marketplace, the address gate, and the controls behind them
Audience	Anyone with a browser. No development experience assumed
Duration	About 45 minutes for the browser tests; 15 more for the database checks
Prerequisite	A deployed stack with the demo dataset loaded

This page is intentionally blank.

Contents

1. Before You Start

1.1 What you need

Item	Value
The address	`http://<host>:3000/` — type `http://` explicitly. Browsers now upgrade bare hostnames to HTTPS, and the demo does not serve TLS, so omitting it produces `ERR_SSL_PROTOCOL_ERROR` and looks like a network fault
A browser	Anything current. Two windows, or one plus a private window, so two people can be signed in at once
Optional	Shell access to the host, for section 12 onward

1.2 The accounts

Every one of these has the password `demo1234`. They exist to be signed into and they are published in a public repository, so they are worth exactly nothing outside a demonstration.

Sign in as	Role	Fee agreement	Exists to show
`marcus@example.com`	Investor	NOT signed	The gate shut
`ruth@example.com`	Investor	Signed	The gate open — the comparison that matters
`ines@example.com`	Investor	Signed	The second brand (KAVADOO), at concierge pricing
`tom@example.com`	Agent	n/a	An agent sees only their own assignments
`priya@example.com`	Agent	n/a	A second agent, to prove the two are isolated
`dan@example.com`	Staff	n/a	Everything, including internal costs
`jpool2@yahoo.com`	Staff	n/a	Administrator

Marcus and Ruth are the pair to keep in mind throughout. Same role, same permissions, same query. The only difference between them is one timestamp in one data column. Wherever this document asks you to compare two windows, it is asking you to look at the consequence of that timestamp.

1.3 Resetting between runs

Some tests change data — a favourite, a saved search. To start clean:

```
cd /opt/sdi
docker compose down -v      # -v destroys the database volume
docker compose up -d
docker compose logs -f db   # wait for 'ready to accept connections'
```

The `-v` deletes the database. That is correct while everything in it is demonstration data and wrong the moment it is not.

2. How To Use This Document

Each test is numbered, states what to do and what should happen, and then says what it proves. Read that last line. A test whose purpose is opaque gets marked “pass” by a tired person at four in the afternoon, and the whole exercise is then worth nothing.

Result	Means	Do this
Pass	Observed exactly what the right-hand column says	Move on
Fail	Observed something else	Record what you saw, and the URL. Keep going — later tests may explain it
Blocked	Could not run it	Say what stopped you

Section 11 is different from every other section. Those tests are expected to be refused, and a refusal is the pass. If any of them succeeds, stop and report it before doing anything else.

Record results on the sheet in section 14.

3. The Anonymous Visitor

Start signed out. This is what a stranger who found the link sees.

T3.1 The marketplace loads

Do this	This should happen
Open the address in a fresh window or a private window.	Filters across the top, a map on the left, listing cards on the right.
Look at the top-right of the page.	It says **Not signed in**, with a **Sign in** link.

What it proves. *Nothing requires an account to browse. That is the product: the listings are the advertisement.*

T3.2 Addresses are withheld

Do this	This should happen
Read the banner under the filter bar.	It explains that street addresses and exterior photographs are withheld until the $750 platform fee agreement is signed, and that everything else is shown in full.
Read the grey line on any card.	A padlock, then city and state only — *address released after signing*. Never a street number.
Count the cards.	**17 properties.**

What it proves. *The gate is stated in plain words before the visitor runs into it. A gate somebody discovers by surprise reads as a bait-and-switch; one explained up front reads as terms.*

T3.3 Every map pin is approximate

Do this	This should happen
Look at the map.	Every price bubble sits inside a soft dashed ring.
Read the legend, bottom-left.	Two entries: *exact location* and *approximate — address gated*.
Hover a bubble.	The tooltip says *approximate location*.

What it proves. *The coordinates sent to this visitor are deliberately offset by roughly a kilometre. Drawing them as sharp pins would claim a precision the data does not have. Reloading does not move them, so repeated loads cannot be averaged to recover the true point.*

T3.4 Some listings are absent entirely

Do this	This should happen
Note the count from T3.2.	17.
Compare with T4.1 later.	Marcus, signed in, will see 18.

What it proves. *Draft and pending listings are not merely hidden from the page — they are never sent. The database refuses the rows; the browser is never in a position to leak them.*

4. Signing In

T4.1 Marcus signs in — the gate stays shut

Do this	This should happen
Click **Sign in**.	A sign-in form.
Enter `marcus@example.com` and `demo1234`.	You land back on the marketplace. Top-right shows **Marcus Pell**.
Look at the count and at the cards.	**18 properties** — one more than anonymous. Addresses still withheld; pins still rings.

What it proves. *Signing in is not the same as being entitled. Marcus now sees a pending listing an anonymous visitor cannot, and still cannot see a single address. Identity and entitlement are separate questions, answered separately.*

T4.2 A wrong password says nothing useful

Do this	This should happen
Sign out. Sign in with `marcus@example.com` and `wrong`.	A single generic failure message.
Now try `nobody@example.com` and `wrong`.	The same message.
Compare the two messages, and roughly how long each took.	Identical wording, and no obvious difference in speed.

What it proves. *A faster answer for an address that does not exist is an enumeration oracle: it lets somebody discover who has an account. The failure path deliberately does the same hashing work as the success path.*

T4.3 Five wrong passwords lock the account

Do this	This should happen
Enter the wrong password for `ines@example.com` five times.	Five failures.
Now enter the correct password, `demo1234`.	Still refused, and the message mentions the account being locked.

What it proves. *Guessing is bounded. Staff can clear it by setting a new password.*

This test leaves Ines locked. Nothing later needs her; a reset (1.3) clears it.

T4.4 Ruth signs in — the gate opens

Do this	This should happen
Sign out. Sign in as `ruth@example.com` / `demo1234`.	Top-right shows **Ruth Okonkwo**. Still **18 properties**.
Look at the banner.	It now says addresses and exact locations are shown, because the fee agreement is on file.
Look at the cards and the map.	Full street addresses. Sharp pins, no rings.

What it proves. This is the whole product. Ruth and Marcus hold the same role and match the same 18 rows. One settled agreement is the entire difference, and no application code is involved in producing it.

T4.5 Side by side

Do this	This should happen
Open a second window in private mode. Sign in as Marcus there, leaving Ruth signed in in the first.	Two sessions, two identities.
Put them next to each other and compare the same listing.	Same price, same beds, same cap rate, same NOI. One shows a street address and a sharp pin; the other shows a padlock and a ring.

What it proves. *Everything an investor underwrites on is public. Only the thing that identifies the parcel is held back. Nothing on the page is a teaser.*

T4.6 An agent sees only their own book

Do this	This should happen
Sign in as `tom@example.com`.	**Tom Bradbury**.
Note the count.	Around 12 — far fewer, including a draft nobody else can see.
Sign in as `priya@example.com` and compare.	A different, smaller set.

What it proves. *Agents are isolated from each other by row policy, not by a filter in the page. Tom cannot reach one of Priya's listings even by guessing its identifier — see T11.3.*

T4.7 Staff see everything

Do this	This should happen
Sign in as `jpool2@yahoo.com`.	**Jessica Pool**.
Note the count.	**25 properties** — every status, including drafts.

What it proves. *Staff see all three bands: the public figures, the addresses, and the internal acquisition cost and margin that never leave the organisation.*

T4.8 Signing out ends the session

Do this	This should happen
As Jessica, click **Sign out**.	Back to **Not signed in**.
Press the browser Back button.	The page may redraw from cache, but the listings return to the anonymous 17 and addresses are gone.

What it proves. *The session is revoked at the server, not merely forgotten by the browser. A copied cookie is dead the moment sign-out happens.*

5. Filtering

Sign in as Marcus for this section.

T5.1 Each filter narrows

Do this	This should happen
Set **Beds** to 3+.	The count drops; every card shows 3 or more bedrooms.
Add a maximum price of 250,000.	Drops again; no card is above $250,000.
Add a minimum of 1,500 sq ft.	Drops again; nothing under 1,500 sq ft.
Press **Reset**.	Back to 18.

What it proves. *Filters are applied by the database inside what you were already allowed to see. There is no filter value that widens the result — the policy runs first, always.*

T5.2 Sorting

Do this	This should happen
Set **Sort** to *Price: low to high*.	Cards reorder, cheapest first.
Set it to *Cap rate*.	Reorders by cap rate, highest first.

What it proves. *Sort options come from a fixed list. A value that is not on the list is ignored rather than passed through to the database.*

T5.3 The dropdowns only offer what you can see

Do this	This should happen
Open the **City** dropdown as Marcus and note the cities.	Nine or so cities.
Sign in as Tom and open it again.	Noticeably fewer.

What it proves. *Even the filter bar is bounded by policy. If it offered every city, its contents would leak the existence of listings the viewer cannot open.*

T5.4 Filters survive a reload

Do this	This should happen
Apply two filters. Copy the URL.	The URL carries the filter values.
Paste it into a new tab in the same session.	The same filtered result.

What it proves. *A search can be shared or bookmarked. It is still evaluated against whoever opens it — sending Ruth's URL to Marcus gives Marcus his results, not hers. Worth actually trying.*

6. The Map

T6.1 Map and list are linked

Do this	This should happen
Hover a card.	Its bubble highlights.

Do this	This should happen
Hover a map bubble.	Its card highlights.
Click a bubble.	The detail panel opens for that property.

What it proves. *The two halves are one view of one result set, not two independent queries.*

T6.2 Clusters stay clickable

Do this	This should happen
Find a city with several listings, e.g. Cleveland.	The bubbles are spread apart rather than stacked.
Click each bubble in the cluster.	Each opens a different property.

What it proves. *Listings in one city sit within a pixel of each other at this scale. They are nudged apart so each is reachable — honest here because for most viewers these positions are approximate anyway.*

T6.3 The map works without the internet

Do this	This should happen
Note whether the map shows roads and place names.	A real basemap, if the host can reach the tile CDN.
If instead you see a plain grid with priced bubbles, read the caption.	*No basemap available — listings plotted to scale.* Bubbles still priced, clickable and hover-linked.

What it proves. *The map library is progressive enhancement. On a restricted network the geography is still legible, and the page says so rather than showing a grey rectangle.*

7. Property Detail

T7.1 Opening a listing

Do this	This should happen
As Marcus, click any card.	A panel slides in from the right with photographs at the top.
Read the top of the panel.	Price, beds, baths, square feet, year built, and a padlocked city-and-state line.
Press Escape.	It closes.

T7.2 The expense breakdown adds up

Do this	This should happen
Find the **Income and operating expenses** table.	Gross rent, vacancy, management, tax, insurance, maintenance, utilities, HOA, and a net figure.
Add up property tax, insurance, maintenance, and utilities if the row says *paid by owner*.	A number.
Compare with the operating expenses figure on the card, or ask staff for `opex_annual`.	**They match exactly.**

What it proves. The breakdown is generated from the published figure rather than typed alongside it. A detail page whose arithmetic does not close is a page nobody underwrites on twice.

T7.3 Area context

Do this	This should happen
Read the section named for the city.	Median household income, median home price, median rent, rent growth, vacancy, price-to-income.
Find *This rent as a share of local income*.	A percentage.

What it proves. *Regional figures live once per city rather than being copied onto every listing, so there is one place to update when a figure moves. Note the caveat: these are plausible demonstration figures, not an ACS extract, and nothing should be underwritten on them.*

T7.4 The gate, in the detail panel

Do this	This should happen
As Marcus, read the coloured note in the middle of the panel.	It says the address, exact pin and exterior photograph are released on signing, and that the financial detail below is complete.
Open the same listing as Ruth.	The note is green and says the agreement is on file. The address is shown.
Compare the financial sections.	**Identical.**

What it proves. *The gate withholds identity, not analysis. Confirming that the numbers are the same for both is the point of this test.*

8. Photographs

T8.1 Marcus sees three, Ruth sees four

Do this	This should happen
As Marcus, open any listing and count the photographs.	**Three** — interiors.
As Ruth, open the same listing and count again.	**Four.**
Look at what the extra one is.	A front elevation.

What it proves. A photograph of the front of a house identifies it as surely as its street number. So exterior shots are gated on the same rule as the address, and interiors are not. Without this the gate would be reopened through the picture gallery.

T8.2 They are illustrations, and say so

Do this	This should happen
Look at the corner of any image.	The word ILLUSTRATION.

What it proves. *The demo draws its own images rather than using anybody's copyrighted listing photography, and they are deliberately not photo-imitations, so nobody mistakes one for the actual house.*

9. Favourites and Saved Searches

Sign in as Marcus.

T9.1 Marking a favourite

Do this	This should happen
Click the heart on a card.	It fills in.
Look at the top-right counter.	It increments.
Open the same listing and check the button in the panel.	It reads *Saved to favourites*.
Reload the page.	Still filled.

What it proves. *Stored against the person, not the browser.*

T9.2 The favourites list

Do this	This should happen
Click **Favourites**.	Only your favourites; the filter bar dims.
Check the address line on the cards.	Still padlocked, exactly as in the grid.
Click it again.	Back to the full result.

What it proves. *The favourites list is built from the same masked view as the grid. An address hidden in search must not reappear because the listing was favourited — the mask is inherited rather than reimplemented, so the two cannot drift apart.*

T9.3 Favourites are private

Do this	This should happen
Note which listings Marcus has favourited.	A list.
In the other window, as Ruth, open her Favourites.	Hers, not his.

What it proves. *One investor cannot see another's saved list — see T11.2 for the version of this test that tries to force it.*

T9.4 Saving a search

Do this	This should happen
Set beds to 3+ and a maximum price of 300,000.	The result narrows.
Click **Save search** and name it `Cheap 3-beds`.	It appears in the *Saved searches* dropdown.
Press **Reset**.	Back to everything.
Choose the saved search from the dropdown.	The filters repopulate and the same result returns.
Open the dropdown again.	It now shows a run count.

What it proves. *The criteria are stored, not the results, so a saved search re-run next month reflects the market next month. The storable fields are constrained by the database: a saved search is replayed later, possibly by different code, and what cannot be stored cannot be replayed.*

T9.5 Saved searches are private too

Do this	This should happen
As Ruth, open the Saved searches dropdown.	Marcus's saved search is not there.

What it proves. *What an investor is hunting for is their own business. Unlike favourites, staff have no override here either.*

10. Plain-English Search

T10.1 It says what it understood

Do this	This should happen
Type `3 bed duplex in Cleveland under 250k best yield` into the top box and press Search.	A bar appears: *Reading that as duplexes, 3+ bed, in Cleveland, under $250,000, best cap rate first*.
Look at the filter bar.	The controls have been set to match.
Look at the results.	Filtered and sorted accordingly.

What it proves. *The box is never opaque about what it did, and its output is the same criteria the controls produce — not a separate path into the database.*

T10.2 More phrasings

Do this	This should happen
Try `cheapest condos over 1500 sqft`.	Condo, 1,500+ sq ft, cheapest first. **Not** a $1.5m price floor.
Try `single family in Tampa between 200k and 400k`.	A price range, a type and a city.
Try `exactly 3 bedrooms in Memphis`.	Beds pinned at exactly 3, not 3 or more.

What it proves. *The second bullet is a real trap: over 1500 sqft and over 1500 match the same shape, and only the unit tells them apart.*

T10.3 Nonsense is refused, not guessed

Do this	This should happen
Type `something something nonsense` and search.	It says nothing was recognised and suggests an example. The filters do not change.

What it proves. *It does not invent a filter to look useful.*

This is a rules parser, not a language model, and it does not call one. It is here because the shape of the feature — free text in, a bounded set of criteria out — is the part that must be right before a model goes behind it.

11. Tests That Must Be Refused

Every test in this section is expected to fail. A refusal is the pass. If any of them succeeds, stop and report it before running anything else.

T11.1 Reading the base table directly

Do this	This should happen
Sign in as Marcus.	—
Open `http://<host>:3000/api/probe` in the address bar.	A refusal: *permission denied for schema core*.

What it proves. The application is not what withholds the address. This request goes around the marketplace entirely and asks the database directly, and the database refuses. Somebody who found an unguarded query, an API bug, or a direct connection would hit the same wall.

T11.2 Reaching another investor's favourites

Do this	This should happen
Ask staff to run:	Only Marcus's own rows — never Ruth's, even though the query names no person and has no WHERE clause.

What it proves. The restriction is in the table, not in the query. There is no query that returns somebody else's saved list.

T11.3 Guessing another agent's listing

Do this	This should happen
As Tom, open a listing and copy its id from the URL or the detail request.	An id.
Sign in as Priya and request the same id:	—
`http://<host>:3000/api/property?id=<that id>`	**404 Not found** — not *403 Forbidden*.

What it proves. Two things. The refusal comes from the row policy rather than a check in the page. And it says *not found* deliberately: answering *forbidden* would confirm the listing exists to anyone who guessed an id.

T11.4 Reaching a draft listing

Do this	This should happen
As Jessica, find the Irvine listing (108 Fairgrove) and copy its id.	Jessica can see it; it is a draft.
Sign out and request the same id as an anonymous visitor.	**404.**

What it proves. Draft listings are invisible to everyone but staff, by policy.

T11.5 The persona switcher is off

Do this	This should happen
While signed out, open `http://<host>:3000/?persona=jessica`.	Still **Not signed in**, and still 17 listings. Not administrator.

What it proves. The demo persona switcher is genuinely useful for showing Marcus and Ruth side by side, but a dropdown that hands out an admin session must not be reachable by accident. It requires `DEMO_PERSONAS=1` and is off by default.

T11.6 Internal figures stay internal

Do this	This should happen
As Jessica, open `/api/view` and find acquisition cost and margin.	Present.
As Marcus, open the same URL and look for them.	Absent, with a permission error recorded against that part of the payload.

What it proves. Acquisition cost and margin are refused by a column grant — a hard access control list, not a masking rule. Marcus cannot read that column under any query.

12. Checks From the Command Line

These need shell access to the host. They take about fifteen minutes and cover the parts a browser cannot show.

12.1 The standing invariants

```
docker compose exec db psql -U postgres -d sdi \
  -c "SELECT * FROM api.security_invariants()"
```

Zero rows is the pass. This is the single most valuable check in the document. It catches the handful of changes that quietly dismantle the model: granting access to the internal schema, exposing an internal column, switching row security off, creating a view that runs with the wrong privileges, or exposing a dimension the fair-housing register forbids. Run it after every deployment, and wire it into whatever runs nightly.

12.2 The fair-housing assertion

```
docker compose logs web | grep fair-housing
```

What you see	Means
`fair-housing register: 17 dimensions, none exposed as filters`	Pass. The check ran and the filter list is clean
Nothing at all	The container did not get that far. Check whether it is running
A `WARNING` about not reading the register	The check did not run. On an older build this could happen transiently at startup; it should now retry and refuse to start rather than serve unchecked

A protected characteristic, or a proxy for one, offered as a filter or a sort is steering, and the law does not require that anyone intended it. The web tier checks its own filter list against the register in the database and refuses to start on a collision, because serving unchecked filters is worse than being down.

12.3 Where the data-rights register stands

```
docker compose exec db psql -U postgres -d sdi \
  -c "SELECT * FROM api.governance_status" \
  -c "SELECT * FROM gov.uncovered_publication"
```

Column	Expected today	Means
`enforcement_mode`	`advisory`	Publication is not blocked; gaps are reported. Flipping to `blocking` is the go-live gate
`rights_recorded`	3	Synthetic demo data, the empty instrument for the tracked Irvine address, and the operator-supplied photograph
`rights_confirmed`	1	Only the synthetic data, which needs no external agreement
`uncovered_published`	0	Nothing is on public display without a right covering it
`published_total`	24	25 properties, of which the Irvine draft is unpublished
`regulations_without_control`	8	Regimes with no enforcement here. Honest, not an error — see the compliance register

`gov.uncovered_publication` should return no rows. Any row names a published listing and the reason no right covers it — missing, expired, unreviewed, or out of territory.

12.4 The listing-status walkthrough

```
docker compose exec -T db psql -U postgres -d sdi \
  < sql/23_listing_sync_tests.sql
```

Ten steps, printing what happens at each: a listing goes under contract, escrow fails and it returns to market, the feed goes down, the listing is genuinely delisted, an advisory source disagrees, and a status term nobody has mapped arrives. Two results are worth watching for specifically.

Step	What to look for	Why it matters
3	*pending* → *active* on the **first** sighting	Escrow fails perhaps one deal in five. A saleable house shown as unavailable costs enquiries every night it waits, so this direction is acted on immediately
4	Three errors in a row change **nothing**	A feed being down must never look like a market that emptied. An adapter in doubt reports an error, never an absence

Step 7 prints an `ERROR` and that is the pass — it is demonstrating that blocking mode refuses an uncovered publication. The file restores everything it changes.

12.5 The governance walkthrough

```
docker compose exec -T db psql -U postgres -d sdi \
  < sql/27_governance_tests.sql
```

Builds a data right one failing condition at a time — unreviewed, no territory, use not granted — and shows it granting nothing until all four conditions hold. Then shows the same confirmed right refusing to cover a property one state away. That is the licensing breach the schema exists to prevent, and the one where the software would otherwise work perfectly.

13. Known Gaps

Do not raise these as defects. They are recorded, and the reasons are in section 11 of the System Documentation.

You will notice	Why
108 Fairgrove has no photograph	The image file is not committed to the repository, and that directory is built into the web image. Only staff can see the listing anyway — it is a draft
The Irvine listing has no price or room counts	Deliberate. The facts held are the ones in the source listing URL; the rest is licensed content that has not been obtained. Inventing plausible numbers would produce exactly the confident wrong record the schema exists to prevent
No listing status ever changes on its own	No MLS feed is connected, and the worker is not deployed in the current compose file
No password reset	Authentication works; recovery is not built. Staff set a new password
The map has no non-visual equivalent	An accessibility gap, recorded and unaddressed
Gated photographs are ordinary files	The database controls who is told an image's address, not who can fetch it. Adequate for generated illustrations, not for real location-revealing photography

14. Recording Results

Section	Tests	Pass	Fail	Blocked	Notes
3. Anonymous visitor	T3.1–T3.4				
4. Signing in	T4.1–T4.8				
5. Filtering	T5.1–T5.4				
6. The map	T6.1–T6.3				
7. Property detail	T7.1–T7.4				
8. Photographs	T8.1–T8.2				
9. Favourites and saved searches	T9.1–T9.5				
10. Plain-English search	T10.1–T10.3				
11. Must be refused	T11.1–T11.6				
12. Command line	12.1–12.5				

Tested by		Date	
Build / image tag		Host	

Any failure in section 11 is a stop-work item. Everything else can be triaged.